or when a node moves to other location, based on received control packets such as EB or DIO, the node autonomously allocates cells. The first autonomous cell scheduling scheme is Orchestra, introduced by Duquennoy *et al.* [6], that offers two mutually exclusive modes: receiver-based (Orch-RB) and sender-based (Orch-SB). Each node deterministically reserves a single unicast cell from its ID. In Orch-RB, that cell functions as an `Rx` slot for the receiver and as a `Tx` opportunity for sender; in Orch-SB, roles are reversed. A key limitation is contention in Orch-RB (many neighbors target the same `Rx` slot) and queuing in Orch-SB as a node can forward at most one packet to its parent per slotframe) Moreover, both modes rely on fixed allocations, which ignore traffic dynamics.

To alleviate Orchestra's rigidity, Kim *et al.* proposed ALICE [7], which adopts *link-based* scheduling, allocating two distinct unicast cells per parent and child pair for both upward and downward traffic. ALICE also uses link-based `Channel Offset` selection and periodically refreshes all unicast cells via time-varying hashing. Nevertheless, ALICE uses two cells, one uplink and one downlink per slotframe, which can throttle forwarding parents under heavy load. Additionally, because each neighbor-specific reservation implies at least one wakeup per slotframe, radios stay active even during low traffic, increasing energy use.

To address ALICE's static behavior, two adaptive autonomous cell scheduling schemes were proposed *i.e.,* OST [8] and A3 [9], aiming to provision communication cells more responsively. In OST, nodes piggyback buffer occupancy on data frames, and then the receiver nodes estimate neighbor demand and allocate cells accordingly. Since scheduling reacts after load is observed, transmissions are not granted immediately, which increases latency. On the other hand, instead of piggybacking the traffic demand, A3 estimates it and accordingly performs cell scheduling. However, due to traffic prediction, cell scheduling takes longer, increasing cell underutilization, packet latency, and node energy consumption. Furthermore, in both OST and A3, at least one per-neighbor wakeup per slotframe is maintained, similar to ALICE, raising energy consumption under low traffic [10]. The work in [10] attempted to reduce such energy consumption by allowing nodes to transmit their packets in shared cells. However, contention in shared cells may increase packet latency, as discussed in that work. The work in [16], [17] used Reinforcement Learning-based techniques at the root node for scheduling cells, thereby incurring the control packet overhead. Apart from these works, the other autonomous scheduling schemes, such as [18]–[20], mainly focus on autonomously scheduling the *shared minimal cell* for control traffic only, without addressing data scheduling. Table I summarizes the existing works on autonomous cell scheduling in 6TiSCH networks along with our proposed scheme.

TABLE I: Comparison of existing scheduling schemes.

Approach	Scheme	Scheduling Type	Convergence Time	Collision Domain	Packet Overhead	Energy Awareness
Centralized	TASA [3]	Adaptive	Moderate	Moderate	High	No
	MODESSA [13]	Adaptive	Moderate	Moderate	High	No
Distributed	MSF [4]	Adaptive	Moderate	Moderate	High	No
	ASAP [21]	Static	Moderate	Moderate	Low	No
	DeTAS [14]	Adaptive	Moderate	Moderate	High	No
	OTF [5]	Adaptive	Low	Moderate	High	No
	e-OTF [22]	Adaptive	Low	Moderate	High	No
Autonomous	Orchestra SB [6]	Static	-	High	No	No
	Orchestra RB [6]	Static	-	High	No	No
	ALICE [7]	Static	-	High	No	No
	OST [8]	Adaptive	High	Low	Low	No
	TACTILE [18]	Adaptive	-	-	No	No
	TRGB [19]	Adaptive	-	-	No	No
	A3 [9]	Adaptive	High	Low	No	No
	OASA [10]	Adaptive	Low	Low	No	Partially
	EASE	Adaptive	Low	Low	No	Yes

III. Proposed Approach

A. Transmission in Autonomous Shared Cells

When a child node transmits its first packet, or transmits after an interval of two or more slotframes, it schedules to transmit it in the shared cell determined by the parent's `EUI64` address as follows:

$$T_b = mod(\texttt{hash}(\texttt{EUI64}(\mathbf{P}) + ASFN),\ SF_s),$$
$$C_b = mod(\texttt{hash}(\texttt{EUI64}(\mathbf{P}) + ASFN),\ N_c - 1) + 1.$$

Here, $ASFN = \lfloor ASN/SF \rfloor$ reduces the likelihood of recurring *hash function*[1] collisions, as in ALICE [7], and N_c denotes the number of available channels. Since both the sender and receiver use the same hash function with complementary `Tx/Rx` roles, scheduling conflicts are inherently avoided. Note that all child nodes transmitting their first packet must contend for the shared cell. In the case of simultaneous first-packet arrivals, nodes access the shared cell using standard CSMA/CA, which may result in collisions. Unsuccessful nodes remain eligible to contend in subsequent slotframes and re-attempt transmission following the IEEE 802.15.4e backoff procedure, thereby ensuring reliability without additional control overhead. To mitigate the additional delay introduced by a large number of contending nodes, we impose a fairness constraint wherein a node that successfully transmits a packet in a shared cell is prohibited from contending for that shared cell in the subsequent slotframe. This mechanism reduces the contention set over time, thereby lowering collision probability in subsequent slotframes and enforcing fairness in a fully distributed manner. However, after a successful transmission in a shared cell, a node attempts to transmit its remaining packets, if any, using *autonomous dedicated cells*, as described in the next subsection. This shared-cell transmission mechanism maintains the parent's low RDC when traffic is low or when child nodes do not generate packets in every slotframe, unlike sender-based schemes such as Orchestra, ALICE, OST, and A3. Thus, the parent avoids unnecessary energy consumption by keeping its radio off for inactive child nodes in each slotframe. Fig. 3 illustrates the scheduling of such a shared cell in slotframe *i* (*zone* 1), where nodes B and C contend to transmit their initial packets to node A at cell SH_A, (1,1).

B. Transmission in Autonomous Dedicated Cells

As a node can transmit only one packet per shared-cell access, any additional packets are served via an autonomously allocated dedicated cells in the same slotframe (*zone* 2). However, the slot and channel of such cells are deterministically

[1]Choosing the hashing function is implementation-specific; its role is to pseudo-randomize timeslot and channel selection and thus reduce collisions.

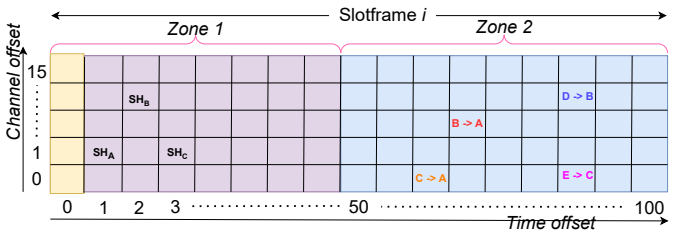


Fig. 3: Scheduling by EASE in Slotframe i.

computed using the EUI64 addresses of both the parent and the child, as follows,

$$T_{slot} = \texttt{mod}\ (\alpha\ \texttt{hash}(\texttt{EUI64}(\mathbf{P}) + \texttt{EUI64}(\mathbf{C}) + ASFN),\ SF_x),$$

$$C_{channel} = \texttt{mod}\ (\alpha\ \texttt{hash}(\texttt{EUI64}(\mathbf{P}) + \texttt{EUI64}(\mathbf{C}) + ASFN),\ N_c - 1) + 1.$$

Here, we divide a slotframe into two *zones* because it is possible that the shared cell with respect to a given parent may appear after the autonomous dedicated cell in the same slotframe. In such a case, the corresponding pair of nodes must wait for the next slotframe to transmit in the autonomous dedicated cell, thereby increasing the latency by *at least* one slotframe length. To address this, we divide the slotframe into two *zones*, as shown in Fig. 3 in slotframe i. In the first *zone*, shared cells are allocated, while in the second *zone*, autonomous dedicated cells are scheduled. This can be achieved using the values of SF_x from $(SF/2 + 1) = SF_s$ to SF, where $SF_x + SF_s = SF$. We partition the slotframe into two equal zones to ensure sufficient non-overlapping shared cells across the network. From a system perspective, the relative sizes of the zones primarily affect the distribution of shared and dedicated cells, but do not significantly alter the total number of available transmission opportunities. As a result, the overall network performance is predominantly governed by the slotframe length, while the impact of zone proportions remains secondary under typical operating conditions. The role of α in the above equations is discussed later.

C. Preventing Starvation with Prediction

Now the question arises: ***how many autonomous dedicated cells should be allocated***? To answer this, EASE incorporates a lightweight traffic predictor based on the Cumulative Sum (CUSUM). Both child and parent nodes maintain local CUSUM estimates of the traffic load. When the parent predicts that the child has packets to transmit, it proactively assigns a dedicated autonomous cell in *zone 2* of slotframe i to that child using the same hashing rule described earlier. Therefore, the child node can directly attempt to transmit in the autonomous cell without contending to transmit in the shared cell. Compared to the exponentially weighted moving average (EWMA) employed in A3, our analysis demonstrates that CUSUM captures bursty traffic more accurately, thereby achieving a higher packet delivery ratio (PDR).

CUSUM-based traffic predictor: Let $D_t \in \mathbb{N}_0$ be the observed demand (packets) on a child–parent link during interval t, and let μ_t be the parent's mean-demand estimate available *before* observing D_t. We can define the residual and a one-sided (upward) CUSUM statistic as follows,

$$z_t = D_t - \mu_t, \tag{2}$$

$$S_{t+1} = \max\{0,\ S_t + z_t\}, \qquad S_0 = 0. \tag{3}$$

With threshold $H > 0$ and tracking coefficient $\rho \in (0, 1)$, the mean is updated as

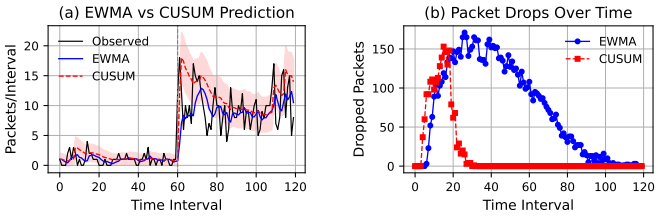
$$\mu_{t+1} = \begin{cases} D_t, & \text{if } S_{t+1} > H, \\ \rho\,\mu_t + (1-\rho)\,D_t, & \text{otherwise,} \end{cases} \tag{4}$$

and the next-interval (in slots) allocated to the child is

$$c_{t+1} = \text{round}\big([\,\mu_{t+1}\,]_+\big), \qquad [x]_+ \triangleq \max\{0, x\}. \tag{5}$$

The residual z_t captures instantaneous demand deviations from the running mean, while the CUSUM statistic S_{t+1} accumulates positive deviations and suppresses negative ones, enabling rapid detection of upward traffic shifts. When $S_{t+1} > H$, the predictor adapts immediately by setting $\mu_{t+1} = D_t$; otherwise, μ_{t+1} is updated via exponential smoothing with weight ρ. The resulting capacity c_{t+1} is obtained by rounding the nonnegative mean to an integer number of slots, ensuring feasibility under bursty traffic.

We conduct a Python-based simulation with 50 IoT nodes organized in an m-ary routing tree to compare EWMA and CUSUM predictors under dynamic traffic. The network transitions from a low-load phase (mean = 1 packet/interval) to a high-load phase (mean = 10 packets/interval), modeling bursty IIoT traffic. As shown in Fig. 4, EWMA adapts slowly to abrupt load changes, resulting in delayed response and higher packet drops, whereas CUSUM rapidly detects the shift, achieving faster convergence, higher prediction accuracy, and significantly lower packet loss. Note that the CUSUM-based predictor incurs constant-time $O(1)$ computation per update and requires only a small number of scalar state variables per child node, resulting in a few bytes of RAM and sub-kilobyte ROM overhead. As a result, it can be efficiently executed on resource-constrained platforms such as the TI CC2650 without imposing significant computational or memory overhead.



(a) Traffic Prediction (b) Packet Drops Over Time

Fig. 4: Comparison of EWMA and CUSUM predictors under identical traffic conditions. The time interval is in minutes.

D. Energy-Aware Slot Allocation using Game-Theory

So far, our methods have been sender-centric, *i.e.,* they help senders transmit their packets efficiently. However, if a relay node has many child nodes that generate bursty traffic

and demands too many slots, such unrestricted forwarding of packets by the node may quickly deplete its battery, potentially disconnecting entire subtrees. To address this, EASE integrates a *non-cooperative gaming* model in which each child decides how many slots to request, while the parent enforces the Radio Duty Cycle (RDC) limit through a congestion price. The game reaches an equilibrium in which the budget is shared fairly according to each child's priority. In brief, to extend network lifetime, relay nodes can operate under a fixed RDC budget in each slotframe, *e.g.,* 20% RDC using a *non-cooperative gaming* model. Note that the border router may assign such budget using application-layer protocols such as CoAP or MQTT or it can be fixed during nodes' deployment.

E. Game-Theoretic Formulation

Consider the set of child nodes $C = \{C_1, C_2, \ldots, C_M\}$, where C_i denotes the i^{th} child associated with a given parent. The parent operates with slotframe length SF and duty-cycle constraint $\delta \in (0, 1)$, yielding a RDC budget of

$$B = \delta SF, \tag{6}$$

which represents the maximum number of RX cells that the parent can schedule within a slotframe while satisfying its duty-cycle constraint. Each child C_i selects an allocation $k_i \geq 0$ (TX→RX cells at the parent) subject to the aggregate constraint

$$\sum_{i=1}^{M} k_i \leq B, \tag{7}$$

ensuring that the total number of allocated cells across all children does not exceed the parent's available RDC budget. If we also consider the individual requirement of cells by each node, then

$$k_i^{\min} \leq k_i \leq k_i^{\max}, \qquad \forall i. \tag{8}$$

These bounds guarantee fairness by enforcing a minimum allocation to avoid starvation and a maximum limit to prevent any single node from monopolizing the available cells. Each child is assigned a positive weight w_i that represents its relative priority. Priority reflects the importance of the data; for instance, in IIoT, a node sending safety-critical alerts gets higher priority than one sending routine status updates. Note that the priority is pre-assigned. Now, the allocation process is modeled as a non-cooperative game

$$G = \{C, (\Psi_i)_{C_i \in C}, (\varphi_i)_{C_i \in C}\},$$

where $\Psi_i = [k_i^{\min}, k_i^{\max}]$ denotes the strategy set of C_i and φ_i its payoff function, which is defined as follows,

$$\varphi_i(k_i, k_{-i}; \lambda) = u_i(k_i) - \lambda k_i, \tag{9}$$

here, the payoff function maintains the trade-off between the acquired transmission cells and the cost imposed by the parent. To stop everyone from demanding too many slots, the parent assigns a price per slot, denoted by λ. The utility function $u_i(k_i)$ is defined as follows,

$$u_i(k_i) = w_i \log(1 + k_i). \tag{10}$$

This logarithmic utility function ensures diminishing marginal returns, meaning that the benefit of acquiring additional cells decreases as k_i increases. It also guarantees strict concavity, which is essential for the existence and uniqueness of the Nash equilibrium, while incorporating node priority through the weight w_i.

F. Solution of the Game

The Nash Equilibrium (NE) is obtained by solving the first-order optimality condition

$$\frac{w_i}{1 + k_i^\star} = \lambda, \tag{11}$$

which yields the unconstrained best response

$$k_i^\star(\lambda) = \frac{w_i}{\lambda} - 1. \tag{12}$$

Imposing the feasibility bounds results in

$$k_i^\star(\lambda) = \Pi_{[k_i^{\min}, k_i^{\max}]}\Big(\max\{0, \tfrac{w_i}{\lambda} - 1\}\Big), \tag{13}$$

with Π denoting projection onto the admissible interval. Then, we can write,

$$D(\lambda) = \sum_{i=1}^{M} k_i^\star(\lambda). \tag{14}$$

where $D(\lambda)$ denotes the aggregate demand of transmission cells from all child nodes. The monotonic decrease of $D(\lambda)$ with respect to λ ensures that increasing the price reduces the total requested cells. So, $D(\lambda)$ is continuous and strictly decreasing in λ, there exists a unique $\lambda^\star$ such that

$$D(\lambda^\star) = B, \tag{15}$$

The above equation denotes a unique equilibrium price $\lambda^\star$ that exactly matches the parent's RDC budget B. This guarantees both feasibility and optimal resource utilization.

Theorem 1. *For any fixed price $\lambda > 0$, the non-cooperative game G admits a unique pure-strategy Nash equilibrium $k^\star = (k_1^\star, k_2^\star, \ldots, k_M^\star)$.*

Proof. For each player i, the strategy set $\Psi_i = [k_i^{\min}, k_i^{\max}]$ is nonempty, compact, and convex; hence, the joint strategy space $\Psi = \prod_{i=1}^{M} \Psi_i$ is also compact and convex. The payoff function $\varphi_i(k_i, k_{-i}; \lambda) = w_i \log(1 + k_i) - \lambda k_i$ is continuous on Ψ. These properties ensure that each player operates within a well-defined and bounded decision space, satisfying the standard conditions required for equilibrium analysis in non-cooperative games.

Moreover, for $k_i > -1$,

$$\frac{\partial^2 \varphi_i}{\partial k_i^2} = -\frac{w_i}{(1 + k_i)^2} < 0, \tag{16}$$

which shows that the payoff function is strictly concave in k_i. This implies that each player has a unique best response for any given strategy of other players. Thus, the game admits a unique NE due to the strict concavity of the utility function combined with the convex strategy space. In brief, since $w_i > 0$, implying that $\varphi_i(\cdot, k_{-i}; \lambda)$ is strictly concave in k_i.

Hence, each player has a unique best response. By the Debreu–Glicksberg–Fan theorem, the game admits at least one NE. The strict concavity of the payoff functions further guarantees that this equilibrium is unique for a given λ. □

G. Realization of Game-Theoretic Model in TSCH

Based on the proposed game-theoretic model, the parent computes the equilibrium allocations q_i, which represent each child node's fair share of transmission opportunities according to its priority. To accommodate event-driven traffic bursts that may temporarily exceed these allocations, a lightweight token-bucket mechanism is employed, augmented with CUSUM-based traffic prediction for early surge detection. Each child C_i is assigned an integer quota q_i and predicted traffic $P_i \geq 0$, subject to the constraint $\sum_{i=1}^{M} q_i \leq B$.

1) At the start of each slotframe, the parent initializes the token counter as

$$\texttt{tokens}_i = \begin{cases} \max(q_i, P_i), & \text{if } \sum_{i=1}^{M} q_i \leq B, \\ \min(q_i, P_i), & \text{otherwise.} \end{cases} \tag{17}$$

2) Upon each packet arrival from child C_i:
   - If $\texttt{tokens}_i > 0$, the packet is acknowledged (ACK sent) and the counter is decremented:
   $$\texttt{tokens}_i \leftarrow \texttt{tokens}_i - 1.$$
   - If $\texttt{tokens}_i = 0$, the parent deliberately withholds the ACK, thereby marking the packet and signaling congestion to the child.

This mechanism ensures that the runtime transmission opportunities of each child remain close to their allocated share q_i or P_i depending on the conditioned mentioned above, while requiring only a simple per-child counter.

H. Numerical Example of our Gaming Model

Let the parent have slotframe length $L = 101$ and duty-cycle $\delta = 0.2$, so the RX budget is $B = \delta L = 20$. Consider $M = 4$ children with priorities $(w_1, w_2, w_3, w_4) = (4, 3, 2, 1)$. If no bounds are active, the equilibrium satisfies

$$\sum_{i=1}^{M} \left(\frac{w_i}{\lambda} - 1 \right) = \frac{\sum_{i=1}^{M} w_i}{\lambda} - M = B,$$

which gives

$$\lambda^\star = \frac{\sum_{i=1}^{M} w_i}{B + M}.$$

With $\sum_i w_i = 4 + 3 + 2 + 1 = 10$, $B = 20$, and $M = 4$, we obtain

$$\lambda^\star = \frac{10}{20 + 4} = \frac{10}{24} \approx 0.4167.$$

The unconstrained equilibrium allocations are then

$$k_i^\star = \frac{w_i}{\lambda^\star} - 1,$$

yielding

$$k_1^\star = \tfrac{4}{0.4167} - 1 \approx 8.6, \quad k_2^\star = \approx 6.2, \quad k_3^\star = \approx 3.8, k_4^\star = \approx 1.4.$$

These allocations satisfy the bounds and sum exactly to $B = 20$. Rounding to integers for slotframe enforcement gives quotas

$$q = (9, 6, 4, 1), \qquad \sum_i q_i = 20.$$

To realize these allocations in TSCH, the parent uses the token bucket mechanism mentioned above.

Algorithm 1 EASE: Energy-Aware Autonomous Scheduling

1: **Input**: Slotframe SF, channels N_c, RDC δ
2: **Calculate**: $ASFN \leftarrow \lfloor ASN/SF \rfloor$, $B \leftarrow \lfloor \delta\, SF \rfloor$
3: **Schedule an Autonomous Shared Cell in Slotframe i:**
   $T_b \leftarrow \mathrm{mod}(\mathrm{hash}(EUI64(P){+}ASFN), SF_s)$
   $C_b \leftarrow \mathrm{mod}(\mathrm{hash}(EUI64(P){+}ASFN), N_c{-}1){+}1$
4: **Schedule an Autonomous Dedicated Cell in Slotframe i:**
   $h \leftarrow \alpha \cdot \mathrm{hash}(EUI64(P){+}EUI64(C){+}ASFN)$
   $T_{slot} \leftarrow SF_s{+}1 + \mathrm{mod}(h, SF_x)$
   $C_{ch} \leftarrow \mathrm{mod}(h, N_c{-}1){+}1$
5: **Prediction:** Apply CUSUM to estimate traffic demand, P_i.
6: **Solve Non-Cooperative Game with Budget B**
   Use gaming model to compute fair quota q_i for each child.
7: **Perform Token-based ACK-transmission:**
   Initialize $tokens_i$, following Eq.17 ;
   Schedule cells in the beginning of a slotframe ;
   Decrement *token* by unity after each packet reception;
   if $tokens_i = 0$, withhold ACK.

I. Scheduling of the Multiple Autonomous Dedicated Cells

The nodes must conditionally schedule either prediction-based or game-theoretic multiple autonomous dedicated cells within a slotframe. However, to mitigate overlapping of shared cells and dedicated in a single slotframe, EASE allocate dedicated autonomous cells in the *zone* 2 of slotframe i as illustrated in Fig. 3. Now the key question arises that is ***how are the dedicated autonomous cells scheduled without conflicts among parent–child pairs***? EASE addresses this as follows. The first ($i = 0$) dedicated autonomous slot $T_{<a,i=0>}$ and channel $C_{<a,i=0>}$ are computed using both the sender's (**S**) and receiver's (**R**) `EUI64` addresses as follows,

$$T_{<a,i=0>} = \texttt{mod}\ (\alpha\ \texttt{hash}(\texttt{EUI64}(\mathbf{R}){+} \texttt{EUI64}(\mathbf{S}) + i + ASFN),\ SF_x),$$

$$C_{<a,i=0>} = \texttt{mod}\ (\alpha\ \texttt{hash}(\texttt{EUI64}(\mathbf{R}){+} \texttt{EUI64}(\mathbf{S}) + i + ASFN),\ N_c - 1) + 1.$$

Here, the variable *i* tracks the number of already transmitted packets, while α is related traffic direction *i.e.,* upward or downward. These ensure that successive autonomous cells are allocated correctly. Since both sender and receiver apply the same hashing function, scheduling conflicts are avoided. However, the sender and the receiver schedule the dedicated autonomous cells at the beginning of a slotframe by sorting them by `slot offset` as the hash function generates pseudo-random cells between SF_s and SF_x. Algorithm 1 shows the steps of our proposed scheme EASE.

Downward Traffic: This refers to commands or queries originating from the root or cloud and directed toward IoT end devices, such as actuators and sensors (e.g., requesting a temperature reading or switching off a motor). In this